

3. Кэширование

3.1 Что кэшировать

Тип данных	Содержимое	Сервис	Критичность
Правила кампаний	Таргетинговые условия, ставки, приоритеты, креативные шаблоны	Сервис ставок, Сервис рекламных кампаний	Высокая – отсутствие приведёт к невозможности проведения аукциона
Бюджеты кампаний	Общий и суточный остаток бюджета	Сервис рекламных кампаний	Высокая – неактуальные данные → overspend или недополучение дохода
Профили пользователей	Сегменты аудитории, frequency capping, история показов	Сервис ставок, Сервис рекламных кампаний	Высокая – влияет на таргетинг и частоту показов
Баннеры	HTML/JS-код, изображения, ссылки (или ссылки на CDN)	Сервис выдачи	Средняя – при отсутствии используется fallback или генерация на лету

Размещение Redis

- Redis Cluster размещается в том же датацентре (или availability zone), что и основные сервисы RTB-пути (Bidding Ingress, Auction Engine), чтобы минимизировать сетевую задержку (< 1 мс).
- Используется Redis Cluster с несколькими мастер-нодами и репликами (минимум 3 мастера, каждый с репликой) для обеспечения высокой доступности и горизонтального масштабирования.
- Размер кэша начально — 64 ГБ памяти на каждый инстанс кластера, с возможностью расширения.

3.2 TTL и стратегия инвалидации

Тип данных	Рекомендуемый TTL	Обоснование
Счётчики частоты (hot counters)	5–15 минут	Данные быстро устаревают; пересчёт лимитов делается по событиям из Kafka
Правила аукциона, лимиты кампаний	1–5 минут	Должны быть свежими; частые обновления через события
Карточки кампаний (общие данные)	10–30 минут	Баланс между актуальностью и нагрузкой на БД

3.3 Стратегии инвалидации

По событию (Event-driven) (рекомендуется для ставок и лимитов):

- При изменении лимита/статуса в PostgreSQL, публикуется соответствующее событие в Kafka.
- Consumer обновляет/удаляет соответствующие ключи в Redis. Это даёт согласованность без долгих TTL.

По истечению времени жизни (TTL-based): для справочников и других данных, где допустима задержка.

Явная инвалидация (Explicit invalidation): при ручных изменениях или действий администраторов (например, «остановить кампанию») сразу удаляем/обновляем ключи.

Также возможна комбинация методов: для критических данных (лимиты, счётчики) использовать события + короткий TTL как страховку. Для справочников — события или длинный TTL.

3.4 Прогрев кэша (cache warming)

Прогрев нужен, чтобы избежать «холодного старта» и всплесков нагрузки на БД при рестарте Redis или деплое.

Подход 1: фоновый воркер (background warmer)

- При старте Redis запускается воркер, который делает выборки «горячих» кампаний (по количеству показов, по бюджету, по активности).
- Данные загружаются в Redis с теми же ключами и TTL, что и в runtime.
- Пример: топ1000 активных кампаний + топ500 DSP.

Подход 2: прогрев по трафику (ondemand + prefetch)

- Bidding Service при старте делает лёгкие выборки последних активных кампаний и заранее кладёт их в кэш.
- Можно использовать скользящее окно «последние N часов» по событиям из Kafka.